

Аннотация

Работа посвящена снижению энергопотребления умножителя Бута radix-4 в адаптивных цифровых фильтрах за счёт вынесенного кодировщика, дополненного методом перекодировки для увеличения числа нулевых частичных произведений. Предложена архитектура, в которой блок кодировки Бута и перекодировщик выносятся за пределы умножителя и выполняются однократно при обновлении коэффициента фильтра. Архитектура реализована на языке Verilog и верифицирована функционально. Площадь умножителя не возросла — предлагаемый вариант даже на 1% меньше базового. Потребляемая мощность в реалистичном сценарии снижена на 4,4% при отношении сигнал/шум 52 дБ и нулевом систематическом смещении — что соответствует требованиям большинства задач цифровой обработки сигналов. Дополнительно показано, что эффективность метода монотонно растёт с разрядностью умножителя — подтверждено на разрядностях 8, 16 и 20 бит. Маршрут измерений построен целиком на открытых инструментах: Yosys, Icarus Verilog, OpenSTA, библиотека NanGate45 (45 нм).

Ключевые слова: умножитель Бута, приближённые вычисления, перекодировка, адаптивные FIR-фильтры, энергопотребление.

Оглавление

Аннотация	1
Обозначения и сокращения	3
Введение	4
1 Предметная область	6
1.1 Умножитель Бута radix-4	6
1.2 Адаптивный FIR-фильтр	7
1.3 Приближённые вычисления в задачах ЦОС	8
1.4 Обзор существующих решений и методов оптимизации	8
2 Метод перекодировки для создания нулевых частичных произведений	11
2.1 Алгоритм перекодировки	11
2.2 Реализованные режимы перекодировки	12
2.3 Статистический анализ доли нулевых строк	13

3	Архитектура с вынесенным перекодировщиком	14
3.1	Проблемы прямого применения метода	14
3.2	Проблема размещения перекодировщика внутри умножителя	14
3.3	Предлагаемая архитектура	15
4	Реализация и методика измерений	17
4.1	RTL-реализация	17
4.2	Программный перекодировщик	17
4.3	Маршрут синтеза и анализа мощности	17
5	Результаты	20
5.1	Площадь	20
5.2	Мощность	20
5.3	Точность	22
5.4	Компромисс мощность/точность	24
5.5	Масштабируемость по разрядности	24
5.5.1	Теоретическая зависимость	24
5.5.2	Экспериментальное подтверждение	25
	Заключение	28
	Список использованных источников	30

Обозначения и сокращения

ЦОС	цифровая обработка сигналов
FIR	finite impulse response — фильтр с конечной импульсной характеристикой
LMS	least mean squares — алгоритм наименьших средних квадратов
RTL	register transfer level — уровень регистровых передач
VCD	value change dump — формат записи активности сигналов
NMED	normalized mean error distance — нормированное среднее расстояние ошибки
MRED	mean relative error distance — среднее относительное расстояние ошибки
SNR	signal-to-noise ratio — отношение сигнал/шум
ED	error distance — расстояние ошибки
ER	exact recoding — точная перекодировка
AR	approximate recoding — приближённая перекодировка
PP	partial product — частичное произведение

Введение

Умножители занимают центральное место в задачах цифровой обработки сигналов (ЦОС): фильтрации, преобразования Фурье, адаптивных систем управления и нейросетевого инференса. В современных цифровых схемах умножитель является одним из наиболее затратных блоков как по занимаемой площади кристалла, так и по потребляемой мощности. Это особенно критично в приложениях реального времени, таких как адаптивные FIR-фильтры: в N -тапном фильтре умножение выполняется на каждом тапе при каждом входном отсчёте, и умножитель доминирует в энергобюджете системы.

В задачах ЦОС зачастую не требуется абсолютная математическая точность вычислений, поскольку сами входные сигналы неизбежно содержат шум. Это открывает путь к использованию энергоэффективных приближённых алгоритмов.

Базовым методом оптимизации умножителей служит алгоритм Бута radix-4 [1]: он вдвое сокращает число частичных произведений по сравнению с прямым двоичным умножением. Zhu и соавт. [11] предложили метод, дополнительно увеличивающий долю нулевых частичных произведений за счёт перекодировки соседних кодов Бута. Нулевые строки в матрице частичных произведений, полученные в результате перекодировки коэффициентов Бута приводят к меньшему числу переключений в модуле финального сложения, что напрямую снижает динамическую мощность.

Тем не менее прямое применение этого метода в адаптивном FIR-фильтре сопряжено с рядом архитектурных препятствий, подробно рассмотренных в настоящей работе. Предлагаемое решение обходит эти препятствия путём выноса блока перекодировки за пределы умножителя с амортизацией его стоимости по числу умножителей в одном фильтре.

Цель работы — снизить энергопотребление умножителя Бута в адаптивном цифровом фильтре, применив методы перекодировки с добавлением нулевых кодов.

Задачи:

- 1) Адаптировать алгоритм перекодировки [11] к параллельной архитектуре умножителя.
- 2) Предложить архитектурное решение, реализующее преимущества перекодировки в сценарии адаптивного фильтра.
- 3) Реализовать предложенную архитектуру на языке описания аппаратуры Verilog.
- 4) Построить методику измерения площади и мощности на общедоступных инструментах (45 нм).

- 5) Оценить точность приближённых режимов по метрикам NMED, MRED, SNR на статистике 10^6 пар.

1. Предметная область

1.1. Умножитель Бута radix-4

Алгоритм Бута radix-4 [1] основан на разбиении n -битного множителя A (в дополнительном коде) на перекрывающиеся группы по 3 бита с шагом 2:

$$(a_{2i+1}, a_{2i}, a_{2i-1}), \quad i = 0, 1, \dots, n/2 - 1, \quad a_{-1} \triangleq 0.$$

Каждая тройка кодируется одним значением из набора $M_i \in \{-2, -1, 0, +1, +2\}$ и представляется тремя управляющими значениями:

- `neg` — знак ($M_i < 0$);
- `one` — $|M_i| = 1$;
- `two` — $|M_i| = 2$.

Итого получается $n/2$ кодов Бута — вдвое меньше частичных произведений (РР), чем при прямом умножении. Каждый код порождает одну строку в матрице РР:

$$P_i = M_i \cdot B \cdot 2^{2i},$$

где B — второй операнд, а множитель 2^{2i} отражает вес i -й группы: поскольку группы сформированы с шагом 2, каждый следующий код Бута соответствует сдвигу множителя на 2 разряда влево, то есть умножению на $4^i = 2^{2i}$. Итоговое произведение получается суммой $\sum_i P_i$ в дереве Уоллеса или Дадды.

Нулевой код Бута ($M_i = 0$) даёт строку $P_i \equiv 0$: она не вносит вклада в сумму и не создаёт переключений сигналов в суммирующем дереве. Именно эта особенность является основой метода перекодировки, рассматриваемого в главе 2.

В данной работе рассматривается 16-битный умножитель в дополнительном коде, дающий $n/2 = 8$ кодов Бута и 32-битный результат. Схема его 8-битного аналога — для упрощения восприятия и компактности рисунка — представлена на рисунке 1; 16-битный вариант устроен аналогично, но содержит вдвое больше кодировщиков, генераторов частичных произведений и строк суммирующего дерева.

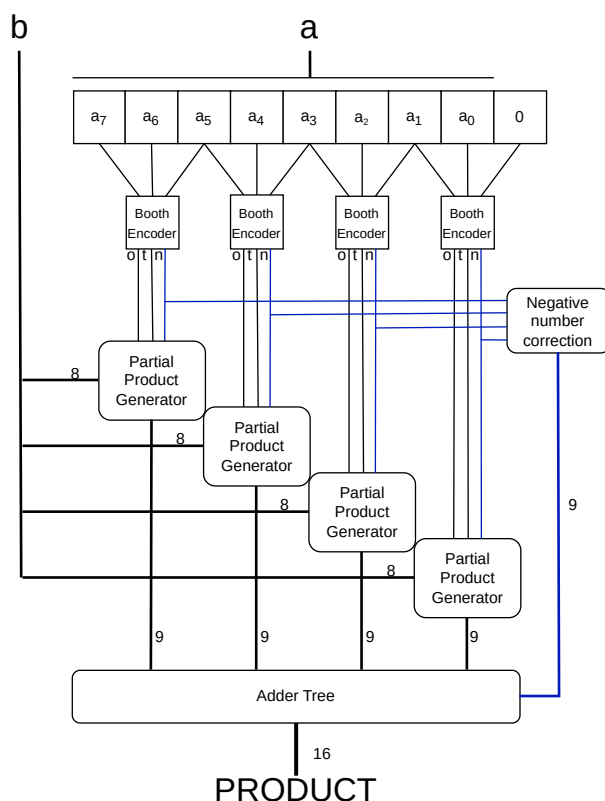


Рисунок 1 — Схема умножителя Бута radix-4. Пример для восьмибитных чисел.

1.2. Адаптивный FIR-фильтр

N -тапный адаптивный FIR-фильтр вычисляет выходной сигнал по формуле:

$$y[n] = \sum_{k=0}^{N-1} w_k[n] \cdot x[n - k], \quad (1.1)$$

где $x[n]$ — входной отсчёт, а $w_k[n]$ — коэффициенты фильтра, обновляемые адаптивным алгоритмом (например, LMS) [14].

Ключевая особенность структуры (1.1) — асимметрия темпов обновления двух операндов каждого умножителя:

- входной отсчёт $x[n]$ меняется каждый такт;
- коэффициент $w_k[n]$ обновляется редко — раз в $10-10^3$ тактов, в зависимости от скорости сходимости алгоритма. Данная асимметрия является ключевым ресурсом для предлагаемой архитектурной оптимизации.

1.3. Приближённые вычисления в задачах ЦОС

Обоснование применимости приближённых вычислений в ЦОС опирается на следующее наблюдение: точность, необходимая для вычислений, не должна превышать точность исходных данных. Сигналы в прикладных системах изначально зашумлены, например это может быть шум квантования записывающего устройства, если речь идет про звук, или шум, добавляющийся при прохождении сигнала через канал связи. Типичный диапазон SNR входного сигнала:

- телефонная речь: 30–40 дБ;
- аудио высокого качества: 50–70 дБ;
- радиолокационный сигнал: 10–40 дБ.

Стандартное 16-битное арифметическое умножение обеспечивает точность около 96 дБ — существенно выше уровня шума в перечисленных приложениях. Иными словами, младшие биты результата умножения несут информацию о сигнале с точностью, которая уже скрыта за уровнем шума: отличить «правильный» результат от «приближённого» на фоне этого шума невозможно. Таким образом, смысла в арифметической точности выше уровня шума сигнала нет, и можно намеренно допустить небольшую арифметическую погрешность, не превышающую этот уровень.

1.4. Обзор существующих решений и методов оптимизации

За последние десятилетия для оптимизации умножителей было предложено множество архитектурных и алгоритмических решений, которые можно разделить на точные и приближённые методы.

Точная оптимизация умножителей Бута. Прямое двоичное умножение генерирует n частичных произведений для n -битных операндов, что требует большого дерева сложения (например, Уоллеса или Дадды) и приводит к значительной динамической мощности из-за высокой активности переключений. Алгоритм Бута [1] и его модифицированная версия radix-4 (MacSorley, 1961) [2], математические основы которой были рассмотрены в разделе 1.1, стали промышленным стандартом знакового умножения: перекодировка radix-4 позволяет обрабатывать тройки бит множителя с перекрытием в один бит, что вдвое сокращает число частичных произведений. Среди современных работ по точной оптимизации можно выделить технологию избыточного кодирования без компенсационных битов (non-redundant radix-4 signed-digit encoding), предложенную Tsoumanis и др. [3], и метод параллельной генерации частичных произведений Cui и др. [4], устраняющий дополнительные компенсационные цепи. Однако потенциал точных методов ограничен: сохраняя полную математическую точность результата, они оптимизируют лишь структуру схемы, и достижи-

мый ими выигрыш по мощности близок к исчерпанию. Дальнейшее существенное снижение мощности возможно лишь при отказе от полной точности результата.

Приближённые вычисления в умножителях Бута. Парадигма приближённых вычислений опирается на естественную толерантность многих приложений ЦОС к малым ошибкам [5]. Это особенно характерно для систем беспроводной связи, где сигнал изначально искажён шумом канала и замираниями: эквализация каналов, детектирование в MIMO-системах, демодуляция и декодирование с мягким решением, обработка поднесущих OFDM. Аналогичная устойчивость к небольшой вычислительной погрешности свойственна обработке звука и алгоритмам машинного обучения. Среди приближённых умножителей Бута можно выделить два направления: модификацию кодировщика и модификацию компрессоров суммирующего дерева. К первому относится умножитель Venkatachalam и др. [6], заменяющий аппаратно сложные коэффициенты ± 2 на ± 1 , и гибридное низкоразрядное кодирование (HLR-BM) Waris и др. [7], аппроксимирующее нечётные множители ± 3 алгоритма Бута radix-8 ближайшими степенями двойки. К этому же направлению примыкает схема сокращения декодеров (Decoder Reduction Approximation, DRA) Haider и др. [8]: она уменьшает число декодеров Бута с $N/2$ до $N/4$, сохраняя лишь наиболее значимые ненулевые коды по их вкладу в результат, и превосходит приближённые логарифмические умножители на задаче инференса нейросети. Ко второму направлению относятся упрощённые компрессоры 4-2 Park и др. [9] и близкий подход на основе мажоритарной логики Zhang и др. [10].

Особый интерес представляет подход, предложенный Zhu и др. [11]: вместо усечения операндов или модификации компрессоров авторы предлагают перекодировку соседних коэффициентов Бута с целью намеренного увеличения доли нулевых частичных произведений. При появлении нулевого кода умножитель может пропустить сложение (в последовательной реализации) или избежать переключения логических вентилях (в параллельной реализации) — именно этот метод взят за основу настоящей работы, так как он обеспечивает управляемый компромисс между точностью и энергопотреблением.

Приближённые умножители в адаптивных фильтрах. Применение приближённых умножителей в адаптивных КИХ-фильтрах (например, на основе алгоритма LMS) сопряжено с дополнительными рисками: внедрение приближённых вычислений непосредственно в петлю обратной связи может повлиять на сходимость алгоритма адаптации, что требует отдельного анализа [12]. Большинство существующих работ, включая Esposito и др. [12], идут по пути простой замены стандартного умножителя внутри фильтра на приближённый (drop-in replacement). Такой подход игнорирует фундаментальное временное свойство адаптивных фильтров — описанную в разделе 1.2 асимметрию темпов обновления операндов [13].

Таким образом, анализ существующих подходов показывает, что большинство авторов оптимизируют умножители Бута изолированно, не учитывая специфику их работы в составе систем реального времени. Применение метода увеличения нулевых частичных произведений [11] в адаптивных фильтрах обладает потенциалом для снижения энергопотребления. Однако прямое использование этого метода невозможно без системной переработки архитектуры, учитывающей разницу в частоте обновления операндов. Разработка такой архитектуры и является фокусом данной работы.

2. Метод перекодировки для создания нулевых частичных произведений

2.1. Алгоритм перекодировки

Авторы [11] вводят вспомогательную переменную x и предлагают заменить пару соседних кодов Бута M_{2i+1}, M_{2i} парой K_{2i+1}, K_{2i} по формулам:

$$K_{2i+1} = M_{2i+1} - x, \quad K_{2i} = M_{2i} + 4x. \quad (2.1)$$

Данное преобразование сохраняет суммарный вклад пары в произведение:

$$K_{2i+1} \cdot 4^{2i+1} + K_{2i} \cdot 4^{2i} = (M_{2i+1} - x) \cdot 4^{2i+1} + (M_{2i} + 4x) \cdot 4^{2i} = M_{2i+1} \cdot 4^{2i+1} + M_{2i} \cdot 4^{2i},$$

поскольку $4 \cdot 4^{2i} = 4^{2i+1}$. Полагая $x = M_{2i+1}$, получаем $K_{2i+1} = 0$ — код первой пары обнуляется, а второй принимает значение $K_{2i} = M_{2i} + 4 M_{2i+1}$.

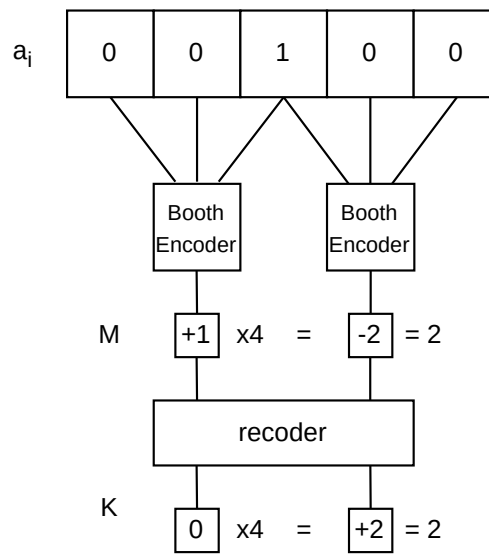


Рисунок 2 — Пример точной перекодировки (ER): $M_{2i+1} = +1, M_{2i} = -2$ переходят в $K_{2i+1} = 0, K_{2i} = +2$

Перекодировка срабатывает, если пятибитный паттерн $a_{2i+3} a_{2i+2} a_{2i+1} a_{2i} a_{2i-1}$ принадлежит одному из наборов, приведённых в таблице 1.

Таблица 1 — Паттерны перекодировки: значения кодов Бута до и после преобразования

Режим	Паттерн	M_{2i+1}	M_{2i}	K_{2i+1}	K_{2i}	ED
ER	00100	+1	-2	0	+2	0
ER	11011	-1	+2	0	-2	0
AR	00101, 00110	+1	-1	0	+2*	+1
AR	11001, 11010	-1	+1	0	-2*	-1

*Точное значение $K_{2i} = \pm 3$ выходит за диапазон кодов Бута и округляется до ± 2 ; разность 1 составляет погрешность ED.

В точном режиме (ER) перекодировка обнуляет K_{2i+1} без ошибки. В приближённом режиме (AR) к этому добавляются ещё 4 паттерна — погрешность составляет $ED = \pm 1$ на одно частичное произведение.

2.2. Реализованные режимы перекодировки

Во всех пяти режимах распознаётся один и тот же набор из шести паттернов: два точных (ER: 00100, 11011) и четыре приближённых (AR: 00101, 00110, 11001, 11010). Различие между режимами состоит в том, в каких позициях коэффициента разрешено применять AR.

При 16-битном коэффициенте алгоритм Бута radix-4 образует 8 пар с индексами $i = 0$ (наименее значимая) до $i = 7$ (наиболее значимая). Ошибка приближённого перекодирования на паре i добавляет погрешность $\pm B \cdot 4^i$ к итоговому произведению: чем старше пара, тем крупнее последствия ошибки. Точная перекодировка (ER) разрешена в любой паре во всех режимах. Приближённая (AR) ограничивается только несколькими младшими парами, которых становится больше с ростом номера режима:

- **exact** — AR не применяется ни в одной паре; результат математически точен.
- **approx_1** — AR разрешена в парах $i < 2$ (2 младшие пары, биты коэффициента a_3-a_0).
- **approx_2** — AR разрешена в парах $i < 4$ (4 младшие пары, биты a_7-a_0).
- **approx_3** — AR разрешена в парах $i < 6$ (6 младших пар, биты $a_{11}-a_0$).
- **approx_4** — AR разрешена во всех 8 парах.

На каждое отдельное срабатывание AR погрешность составляет $ED = \pm 1$ относительно данной пары. Однако суммарная погрешность произведения растёт с номе-

ром режима: более высокие режимы допускают ошибки в старших парах, вес которых 4^i существенно больше.

Важное конструктивное свойство: паттерны 00101/00110 дают $ED = +1$, а 11001/11010 — $ED = -1$. Эти паттерны противоположны, и при случайных данных встречаются примерно одинаково часто. Поэтому систематическое смещение результата близко к нулю, и погрешность не накапливается при длительной работе.

2.3. Статистический анализ доли нулевых строк

Эффективность метода зависит от того, как часто паттерны перекодировки встречаются в реальных данных. На рисунке 3 показана средняя доля нулевых строк матрицы PP для каждого режима, оценённая на выборке из 10^6 пар с равномерным и гауссовым распределениями входных данных.

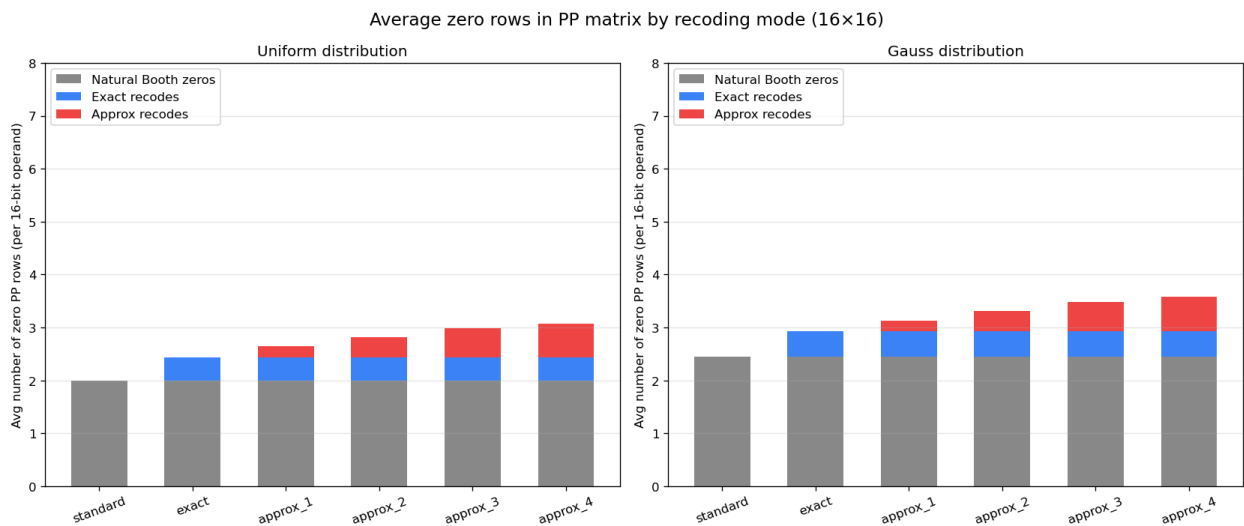


Рисунок 3 — Средняя доля нулевых строк матрицы частичных произведений по режимам перекодировки

Доля нулевых строк монотонно возрастает с уровнем приближения. Для режима `approx_2` она существенно выше, чем для `exact`, что и обеспечивает дополнительную экономию мощности при умеренной погрешности.

3. Архитектура с вынесенным перекодировщиком

3.1. Проблемы прямого применения метода

Метод [11] был разработан для последовательного умножителя: на каждом такте вычисляется одно частичное произведение и прибавляется к общей сумме, если коэффициент Бута оказывается нулевым, то сложение пропускается. Это позволяет экономить тактовые циклы за счёт переменного числа шагов.

Такой подход неприменим в потоковом FIR-фильтре. Каждый умножитель в фильтре обязан выдавать результат каждый такт: переменная задержка потребовала бы межумножительного контроллера готовности, накладные расходы которого перекрыли бы весь выигрыш. Кроме того, потоковые задачи ЦОС требуют фиксированной пропускной способности, несовместимой с последовательной архитектурой.

Схема конфликта показана на рисунке 4: умножители завершают работу за разное число тактов, из-за чего происходит рассинхронизация.

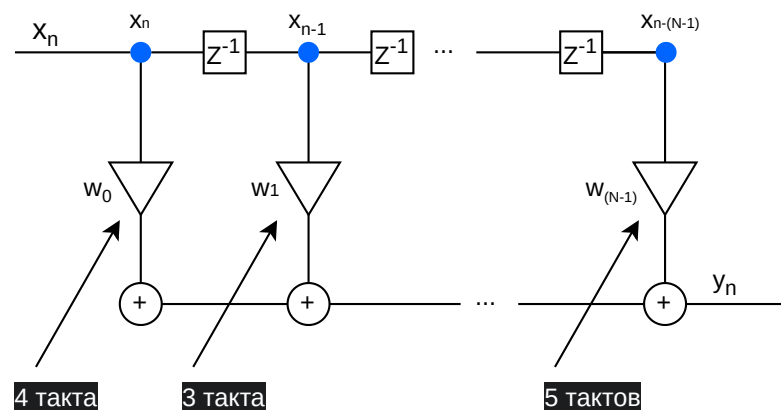


Рисунок 4 — Проблема последовательного умножителя в конвейере FIR-фильтра

Таким образом, метод необходимо адаптировать к параллельному умножителю с фиксированной однотоктной задержкой.

3.2. Проблема размещения перекодировщика внутри умножителя

Очевидным первым шагом кажется размещение перекодировщика (2.1) перед блоком генерации частичных произведений внутри параллельного умножителя. Однако это наталкивается на принципиальное ограничение: алгоритм перекодировки обрабатывает пары кодов Бута последовательно, справа налево.

Причина — перекрывание пятибитных паттернов (рисунок 5): каждый коэффициент участвует в перекодировании дважды, поэтому перекодировка новой пары

невозможна, пока не будет произведена перекодировка предыдущей.

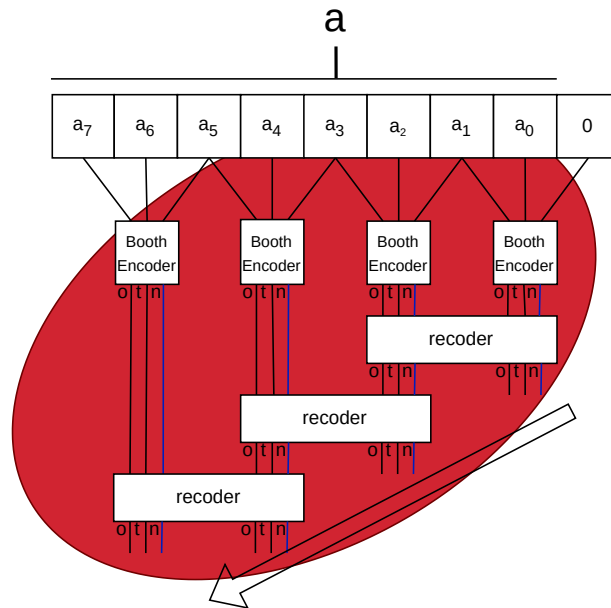


Рисунок 5 — Последовательная зависимость между парами в алгоритме перекодировки. Пример для 8-битного числа.

Возможен вариант, когда рассматриваются только непересекающиеся пары. Но в таком случае число попыток для перекодировки снижается примерно вдвое.

Кроме того, перекодировщик — это дополнительная комбинационная логика, которая сама потребляет динамическую мощность при каждом умножении. В N -тапном фильтре с N умножителями перекодировщик размножился бы в N экземпляров. Таким образом, простое добавление перекодировщика внутрь параллельного умножителя увеличивает мощность, а не снижает её.

3.3. Предлагаемая архитектура

Предлагаемое решение использует асимметрию скоростей обновления, описанную в разделе 1.2. Коэффициент w_k обновляется редко — следовательно, на скорость вычисления этого коэффициента не накладывается жестких временных ограничений. В таком случае можно применять последовательный алгоритм перекодировки для всех пересекающихся пар. Кроме того, один и тот же модуль можно использовать для вычисления коэффициентов сразу нескольких умножителей внутри одного фильтра. Таким образом стоимость вынесенного кодировщика будет разделена на число умножителей, для которых он производит вычисления.

Архитектура строится на четырёх принципах:

- 1) Вынесение. Блок кодировки Бута и перекодировщик выносятся за пределы умножителя в отдельный аппаратный модуль.
- 2) Однократное вычисление. Перекодировка выполняется один раз при каждом обновлении коэффициента — без ограничений реального времени.
- 3) Хранение. Результирующий 24-битный вектор $\{two[7:0], one[7:0], neg[7:0]\}$ записывается в регистр умножителя вместо исходного 16-битного коэффициента.
- 4) Единый RTL. Сам умножитель одинаков для всех режимов точности; выбор режима определяется только содержимым перекодированных коэффициентов, записанных в регистры.

Сравнение базовой и предлагаемой схем умножителя приведено на рисунке 6.

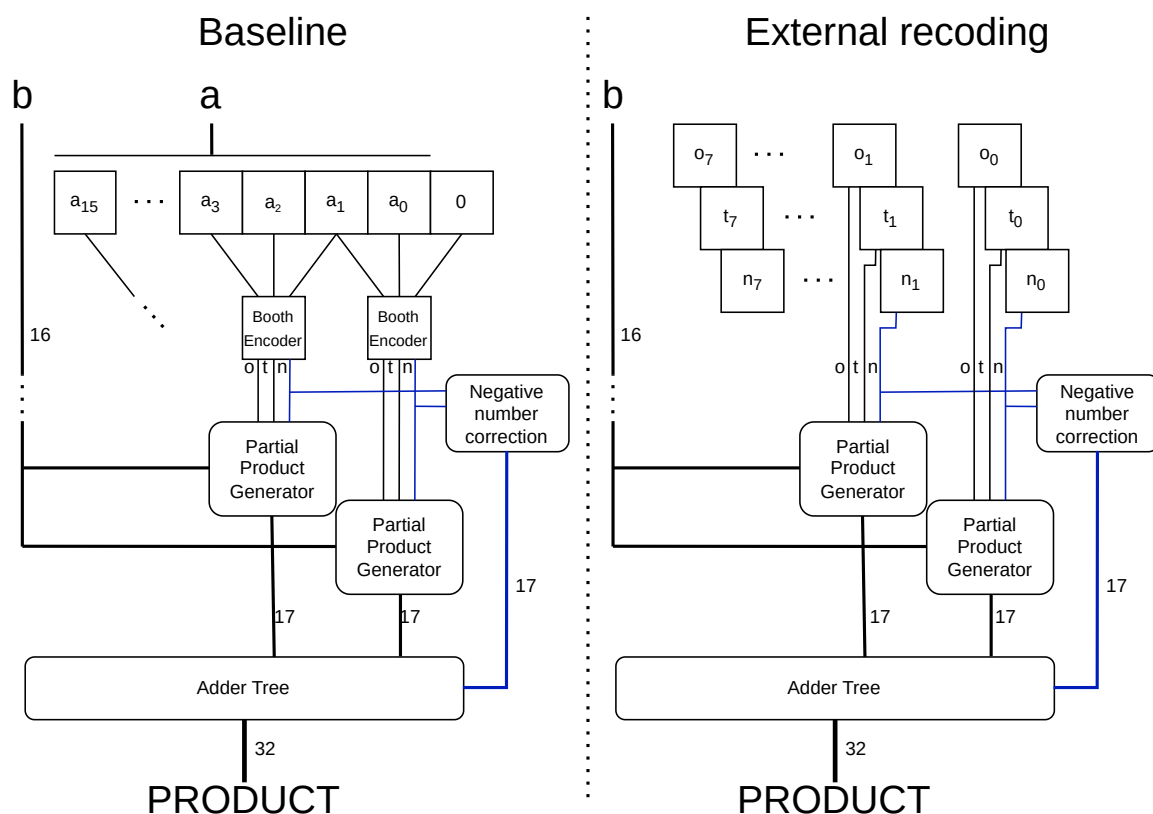


Рисунок 6 — Базовая схема (слева): перекодировщик внутри умножителя. Предлагаемая схема (справа): перекодировщик вынесен наружу, умножитель принимает готовый вектор

4. Реализация и методика измерений

4.1. RTL-реализация

Разработаны два RTL-модуля на языке Verilog:

- **mult_booth** — исходный умножитель: radix-4 Booth с перекодировщиком внутри. Используется как базовый вариант для сравнения.
- **mult_booth_extrec** — предлагаемый умножитель: принимает готовый 24-битный вектор управляющих сигналов {two, one, neg}. Логика кодирования не содержит.

Оба модуля реализуют 16×16 -битное знаковое умножение (дополнительный код), выдают 32-битный результат за один такт и синтаксически идентичны в части итогового суммирующего дерева.

4.2. Программный перекодировщик

Перекодировщик реализован на языке Python. Алгоритм последовательно обрабатывает все 7 пар пятибитных паттернов 16-битного множителя и генерирует 24-битный управляющий вектор. Поддерживаются все пять режимов: exact и approx_1–approx_4.

Результат записывается в регистр аппаратного умножителя через интерфейс конфигурации. При необходимости аппаратной реализации алгоритм может быть перенесён в RTL без изменений в самом умножителе.

4.3. Маршрут синтеза и анализа мощности

Маршрут целиком построен на открытых инструментах и общедоступной библиотеке стандартных ячеек. Этапы показаны на рисунке 7.

- 1) Генерация стимулов. Python-перекодировщик формирует файлы пар (a, b) и соответствующих перекодированных векторов для заданного тестового сценария.
- 2) Синтез. Yosys — открытый инструмент синтеза RTL — транслирует Verilog-описание в gate-level нетлист, отображённый на стандартные ячейки библиотеки NanGate45 — открытой академической библиотеки технологической нормы 45 нм.
- 3) Симуляция. Icarus Verilog — открытый симулятор языка Verilog — прогоняет gate-level симуляцию нетлиста на сформированных стимулах и записывает VCD-файл (value change dump) с активностью всех узлов схемы.

- 4) Анализ мощности. OpenSTA — открытый инструмент статического временного анализа — читает VCD через команду `read_vcd`, аннотирует число переключений на каждую ячейку библиотеки и вычисляет отчёт динамической мощности.

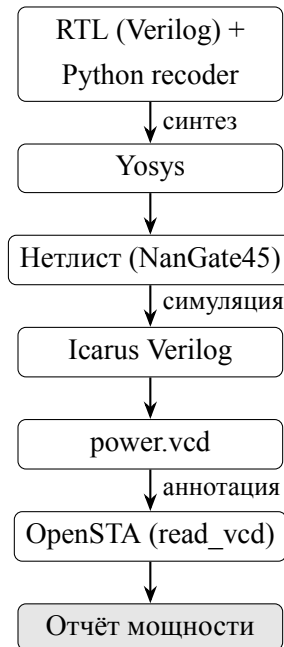


Рисунок 7 — Маршрут синтеза и анализа мощности

Тестовые сценарии. Оценка мощности проводилась в 4 сценариях, образованных комбинацией двух параметров:

- Распределение входного операнда b : равномерное $U[-32768, 32767]$ и гауссово $\mathcal{N}(0, 8000^2)$, усечённое до $[-32768, 32767]$.
- Режим обновления коэффициента a : `fast` — обновляется каждый такт; `slow` — раз в 10 тактов (реалистичный сценарий адаптивного фильтра).

Ограничения методики. Абсолютные значения мощности, полученные описанным маршрутом, носят приближённый характер. Библиотека NanGate45 является академической: параметры её ячеек нормированы иначе, чем в производственных библиотеках (TSMC, Samsung, GLOBALFOUNDRIES и др.). В частности, токи утечки и ёмкости проводников NanGate45 не соответствуют реальным производственным нормам. Кроме того, синтез в Yosys не применяет ряд оптимизаций, доступных в коммерческих САПР (Synopsys Design Compiler, Cadence Genus). По совокупности этих причин абсолютные значения мощности следует интерпретировать как оценку на академической модели; относительные значения — разница между конфигурациями при одинаковых стимулах — остаются достоверными. В производственных технологиях с более тонкими нормами абсолютная мощность, как правило, ниже, а доля пассив-

ной утечки относительно динамической мощности иная; тем не менее качественное соотношение конфигураций сохранится.

Отдельного внимания заслуживает ограничение сценария `slow`. В нем коэффициент обновляется редко — раз в 10 тактов и умножитель длительное время работает с одним и тем же значением a . Чтобы охватить весь 16-битный диапазон коэффициентов $[-32768, 32767]$ (65 536 значений), потребовалось бы прогнать симуляцию суммарной длиной в несколько сотен тысяч тактов, а порождаемый VCD-файл оказался очень большим. Это сделало бы gate-level симуляцию и последующий анализ мощности слишком ресурсоёмкими в рамках доступных вычислительных мощностей. Поэтому в обоих сценариях использовалась случайная выборка из $N = 20\,000$ пар (a, b) , что соответствует лишь части диапазона коэффициентов. Выборка генерировалась с фиксированным зерном генератора случайных чисел, обеспечивая воспроизводимость результатов. Статистические свойства выборки — равномерное и гауссово распределения — достаточно близки к реальным, чтобы относительные оценки мощности между режимами оставались достоверными.

5. Результаты

5.1. Площадь

После синтеза на NanGate45 площадь двух модулей составила:

Таблица 2 — Площадь модулей после синтеза (NanGate45)

Модуль	Площадь, мкм ²
mult_booth (исходный)	2725
mult_booth_extrec (предлагаемый)	2698

Предлагаемый вариант на 1,0% меньше базового — площадь не только не возрастает, но даже немного снижается. Это объясняется действием двух противоположных факторов:

- Уменьшение. Из умножителя полностью убрана логика кодирования Бута — вычисление сигналов *neg*, *one*, *two* из 16-битного коэффициента. Умножитель получает готовый вектор.
- Увеличение. Вместо 16-битного регистра коэффициента используется 24-битный регистр перекодированного вектора ($8 \times 3 = 24$ бит).

Два эффекта взаимно компенсируются, и суммарная площадь умножителя остаётся практически неизменной. Это означает, что предлагаемая архитектура не несёт штрафа по площади кристалла.

5.2. Мощность

Наиболее реалистичный сценарий — *gauss_slow*: входной сигнал имеет гауссово распределение (характерное для сигналов в системах беспроводной связи), а коэффициент фильтра обновляется медленно, что соответствует реальному адаптивному фильтру. Результаты для этого сценария приведены в таблице 3 и на рисунке 8.

В сравнении участвуют три типа конфигураций, которые важно различать:

- *booth* — базовый умножитель Бута с внутренним перекодировщиком (Baseline Booth); эталон для сравнения.
- *extrec_standard* — предлагаемая вынесенная архитектура со стандартными (не перекодированными) коэффициентами: перекодировка выключена.
- *extrec_exact*, *extrec_approx_1*, *extrec_approx_2* — та же вынесенная архитектура с включённой перекодировкой соответствующего режима.

Положительное значение в колонке «Экономия мощности» означает снижение потребления относительно базового умножителя.

Таблица 3 — Потребляемая мощность в сценарии gauss_slow

Конфигурация	Мощность, мкВт	Экономия мощности, %
booth (базовый)	413	—
extrec_standard	412	+0,2
extrec_exact	403	+2,4
extrec_approx_1	399	+3,4
extrec_approx_2	395	+4,4

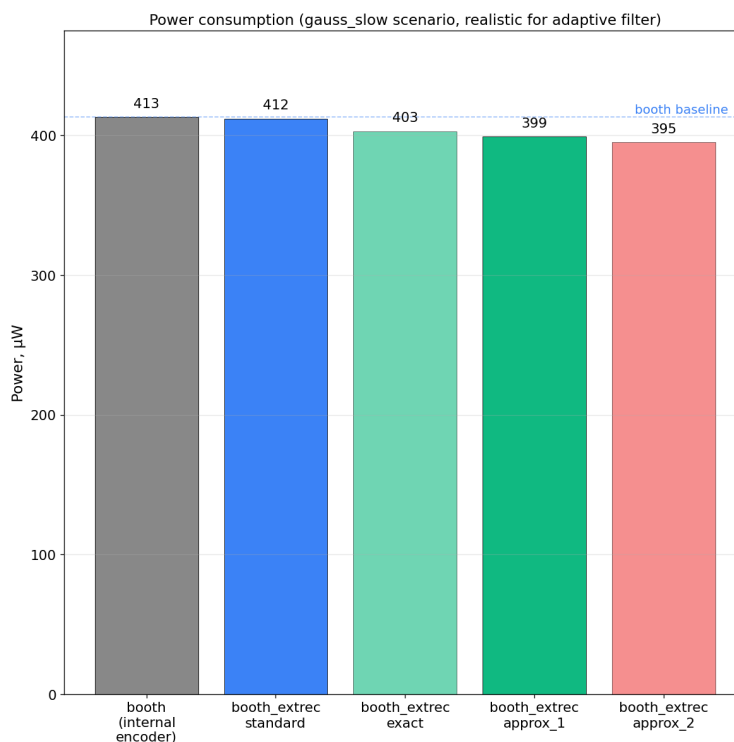


Рисунок 8 — Потребляемая мощность всех конфигураций в сценарии gauss_slow; пунктиром отмечена линия базового умножителя Бута

Отдельного внимания заслуживает конфигурация extrec_standard: это вынесенная архитектура с обычными (не перекодированными) коэффициентами. Её мощность практически совпадает с базовым умножителем (412 против 413 мкВт, разница 0,2%): расширение регистра коэффициента до 24 бит полностью компенсируется устранением логики кодирования. Это измерение намеренно включено в сравнение: оно разделяет два эффекта и подтверждает, что вынесенная архитектура сама по се-

бе мощность не меняет, а всю экономию (до 4,4%) обеспечивают именно нулевые частичные произведения, создаваемые перекодировкой.

Результаты по всем четырём сценариям показаны на рисунке 9. Видно, что сценарий *slow* принципиально важен. Во-первых при более редком обновлении коэффициента модули ожидаемо потребляют меньше мощности относительно быстрого сценария, во-вторых видно, что в этом сценарии предлагаемый модуль проявляет себя лучше. Это еще раз подтверждает факт, что уменьшение мощности вносит именно перекодировка.

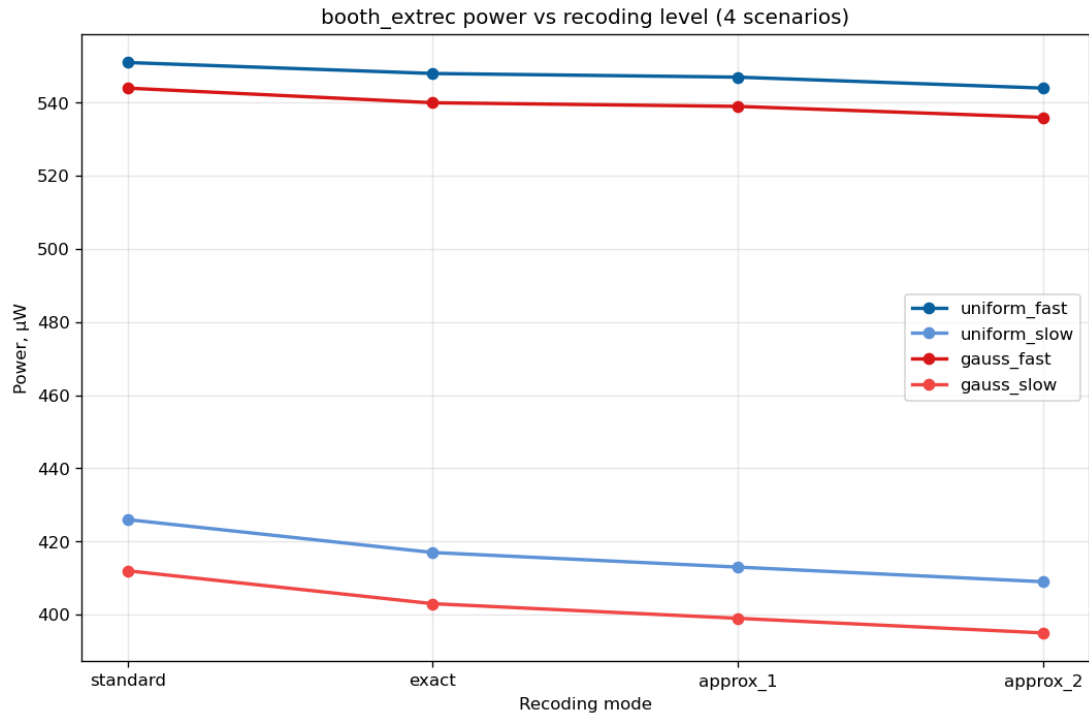


Рисунок 9 — Мощность предлагаемого умножителя по уровням перекодировки в 4 сценариях

5.3. Точность

Точность оценивалась на 10^6 случайных парх операндов с гауссовым и равномерным распределениями по трём метрикам:

- NMED — нормированное среднее абсолютное отклонение результата $\hat{p}$ от точного p : $\text{NMED} = \frac{1}{N} \sum |p - \hat{p}| / p_{\max}$.
- MRED — среднее относительное отклонение, нечувствительное к масштабу операндов: $\text{MRED} = \frac{1}{N} \sum |p - \hat{p}| / |p|$ (по ненулевым p).
- SNR — отношение мощности сигнала к мощности ошибки, дБ; наиболее естественная метрика для задач ЦОС.

Результаты представлены в таблице 4 и на рисунке 10.

Таблица 4 — Метрики точности (гауссово распределение, 10^6 пар)

Режим	NMED	MRED	SNR, дБ
exact	0	0	∞
approx_1	$1,3 \cdot 10^{-6}$	$3,8 \cdot 10^{-4}$	76,7
approx_2	$2,2 \cdot 10^{-5}$	$4,4 \cdot 10^{-3}$	52,3
approx_3	$3,5 \cdot 10^{-4}$	$3,8 \cdot 10^{-2}$	28,1
approx_4	$1,5 \cdot 10^{-3}$	$7,3 \cdot 10^{-2}$	15,6

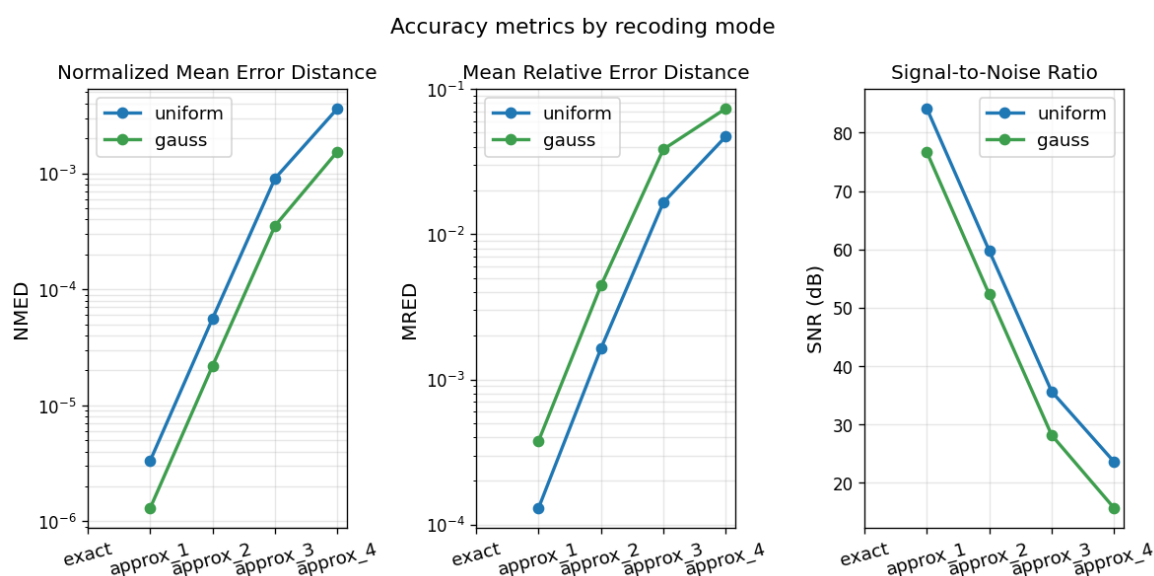


Рисунок 10 — Метрики точности по режимам перекодировки: NMED, MRED, SNR (кривые для равномерного и гауссового распределений)

Режимы approx_3 и approx_4 демонстрируют значительную погрешность (SNR менее 30 дБ) и не рекомендуются для применения в большинстве задач ЦОС. Режимы exact — approx_2 обеспечивают $\text{NMED} < 10^{-4}$ — уровень, считающийся хорошим в контексте приближённых вычислений [11].

Систематическое смещение результата практически равно нулю во всех режимах: ошибки $\text{ED} = +1$ и $\text{ED} = -1$ встречаются с одинаковой частотой и компенсируют друг друга при усреднении. Это исключает накопление систематической погрешности в адаптивном фильтре.

5.4. Компромисс мощность/точность

Таблица 5 — Сводная таблица мощности и точности (сценарий `gauss_slow`)

Режим	Экономия мощности, %	SNR, дБ
standard	+0,2	∞
exact	+2,4	∞
approx_1	+3,4	76,7
approx_2	+4,4	52,3

Режим `approx_2` является наиболее практически привлекательным: экономия 4,4% при $\text{SNR} = 52,3$ дБ. Это значение превышает требования большинства задач беспроводной связи. Нулевое систематическое смещение устраняет риск накопления погрешности.

Режим `exact` рекомендуется для задач, требующих гарантированно точного результата при минимальном энергобонусе.

5.5. Масштабируемость по разрядности

Все рассмотренные выше результаты получены для 16-битного умножителя. Естественно возникает вопрос: как ведёт себя эффективность предлагаемой архитектуры при изменении разрядности операндов? Ниже показано, что теоретически эффективность должна монотонно возрастать с разрядностью, и это подтверждается измерениями на 8, 16 и 20 битах.

5.5.1. Теоретическая зависимость

Предлагаемая архитектура отличается от базового умножителя двумя структурными изменениями: из умножителя убираются $n/2$ кодировщиков Бута и расширяется входной регистр — вместо n -битного коэффициента хранится $3 \cdot (n/2) = 1,5 n$ -битный перекодированный вектор, то есть добавляется $n/2$ разрядов регистра. Обозначив площадь одного кодировщика за x , а одного разряда регистра за y , получим разницу площадей предлагаемого и базового модулей:

$$\Delta S = \underbrace{\frac{n}{2} y}_{\text{доп. регистр}} - \underbrace{\frac{n}{2} x}_{\text{убранные кодировщики}} = \frac{n}{2} (y - x), \quad (5.1)$$

то есть накладные расходы вынесенной архитектуры растут линейно по разрядности n . В то же время площадь и мощность самого умножителя определяются сумми-

рующим деревом: число частичных произведений равно $n/2$, а разрядность каждого — порядка n , поэтому площадь дерева растёт квадратично, $S_{\text{дерева}} \sim n^2$. Отношение

$$\frac{\Delta S}{S_{\text{дерева}}} \sim \frac{\frac{n}{2}(y-x)}{c n^2} \sim \frac{1}{n} \xrightarrow{n \rightarrow \infty} 0$$

убывает с ростом разрядности. Важно подчеркнуть, что соотношение (5.1) предсказывает лишь ограниченность overhead: его абсолютная величина растёт не быстрее линейной, поэтому доля overhead на фоне квадратично растущего умножителя становится пренебрежимо малой. При этом знака формула не меняет — она не предсказывает, что предлагаемый вариант станет меньше базового. Сам по себе эффект перекодировки даёт примерно постоянную (слабо зависящую от разрядности) долю обнуляемых строк PP; а поскольку фиксированная цена вынесения с ростом n амортизируется на фоне умножителя, итоговый выигрыш по мощности относительно базового умножителя не убывает с разрядностью.

Дополнительный аргумент даёт временной анализ. В базовом умножителе кодировщик Бута лежит на критическом пути (его выход управляет генератором частичных произведений), тогда как в предлагаемой архитектуре управляющие сигналы поступают напрямую из регистра. Поэтому extres имеет более короткий критический путь, и с ростом разрядности (где временное давление усиливается) это даёт ему дополнительное преимущество.

5.5.2. Экспериментальное подтверждение

Для проверки гипотезы независимые копии RTL-модулей были синтезированы для разрядностей 8, 16 и 20 бит единым маршрутом синтеза (одинаковый рецепт Yosys/ABC для всех разрядностей — это критично для сопоставимости), после чего для каждой разрядности проведены gate-level симуляция и анализ мощности в сценарии gauss_slow. Результаты приведены в таблице 6 и на рисунке 11.

Таблица 6 — Масштабируемость: площадь и экономия мощности (gauss_slow) по разрядностям

Разрядность	Площадь, мкм ²		Экономия мощн. vs booth	
	booth	overhead	exact	approx_2
8 × 8	723	+1,2%	−0,6%	+2,3%
16 × 16	2725	−1,0%	+2,4%	+4,4%
20 × 20	4135	−1,4%	+2,9%	+4,5%

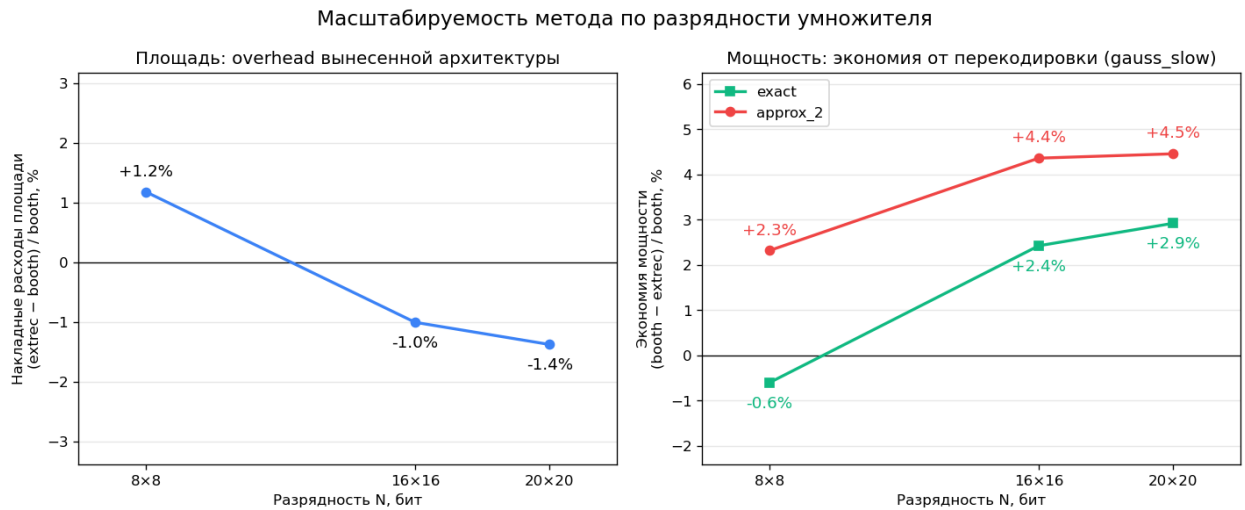


Рисунок 11 — Масштабируемость по разрядности: накладные расходы площади (слева) и экономия мощности от перекодировки (справа)

Измерения согласуются с теоретическим прогнозом в главном: накладные расходы площади малы на всех разрядностях (в пределах 1,5%), и их доля не растёт — линейный overhead тонет в квадратично растущем суммирующем дереве, как и следует из (5.1). Сам знак этого остатка порядка процента самостоятельного смысла не имеет: он определяется деталями технологического отображения логики в АВС. Поэтому по площади корректно утверждать лишь, что предлагаемая архитектура её практически не меняет.

Экономия мощности относительно базового умножителя монотонно возрастает с разрядностью (+2,3% → +4,4% → +4,5% для approx_2; аналогично для exact). Причина — амортизация фиксированной цены вынесения, а не изменение самого эффекта перекодировки: измеренный относительно той же архитектуры (режим standard) выигрыш от перекодировки почти не зависит от разрядности и составляет порядка —4%. На 8 битах его частично перекрывает накладной расход более широкой перекодированной шины — на маленьком умножителе он составляет заметную долю мощности (поэтому точный режим здесь даёт даже небольшой проигрыш, —0,6%); на 16 и 20 битах этот расход пренебрежим, и проявляется почти полный эффект перекодировки.

Монотонность прослеживается и на алгоритмическом уровне. На рисунке 12 приведено среднее число строк матрицы РР, дополнительно обнуляемых перекодировкой, для разрядностей 8, 16 и 20 бит: с ростом n оно монотонно и практически линейно увеличивается во всех режимах (например, для approx_2 — 0,53 → 0,86 → 0,98 строки на операнд). Это подтверждает, что сам метод не теряет эффективности на бóльших операндах. Подчеркнём, что рост относительного выигрыша по мощности обеспечивает в первую очередь убывающий архитектурный overhead (соотношение (5.1)), а не изменение доли обнуляемых строк.

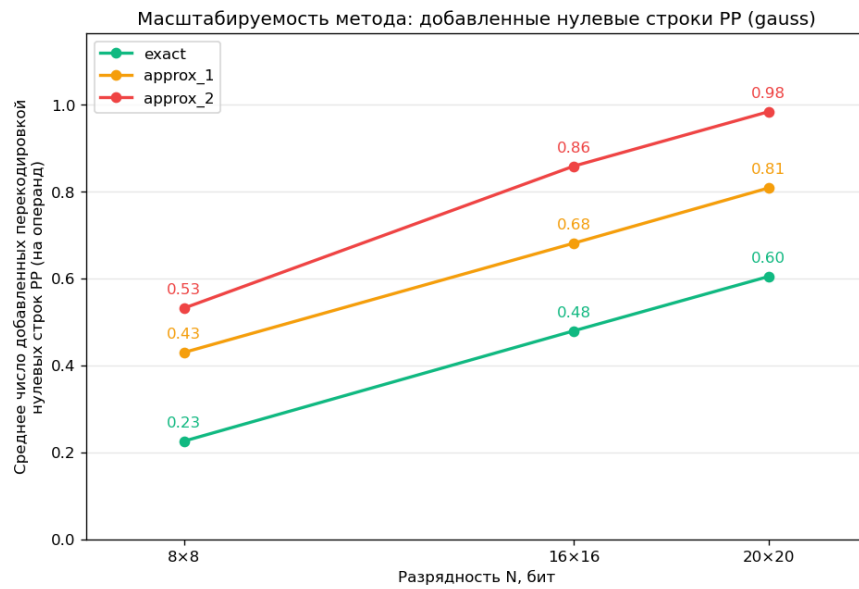


Рисунок 12 — Среднее число добавленных перекодировкой нулевых строк матрицы PP по разрядностям (гауссово распределение)

Таким образом, предлагаемая архитектура не только не теряет, но и усиливает свои преимущества при переходе к более широким операндам, что делает её привлекательной для высокоразрядных задач.

Заключение

В настоящей работе предложена, реализована и верифицирована архитектура умножителя Бута radix-4 с вынесенным перекодировщиком для применения в адаптивных FIR-фильтрах. Основные результаты:

- 1) Выявлены два архитектурных препятствия прямого применения метода перекодирования из статьи [11] в параллельных умножителях FIR-фильтра: несовместимость переменной задержки последовательного умножителя с поточной обработкой и последовательная зависимость алгоритма перекодировки, исключаяющая его распараллеливание.
- 2) Предложена архитектура с вынесенным перекодировщиком, в которой перекодировка каждого коэффициента выполняется однократно при его обновлении, а один такой модуль обслуживает все N умножителей фильтра, последовательно вычисляя перекодированные коэффициенты для каждого тапа — благодаря чему его стоимость амортизируется по числу умножителей.
- 3) Реализованы два RTL-модуля на Verilog: `mult_booth` (исходный) и `mult_booth_extrec` (предлагаемый), поддерживающий пять режимов точности без каких-либо изменений аппаратной части.
- 4) Построен воспроизводимый маршрут измерения площади и мощности целиком на открытых инструментах: Yosys, Icarus Verilog, OpenSTA, библиотека NanGate45.
- 5) Площадь предлагаемого умножителя не возрастает: устранение логики кодирования слегка перевешивает расширение регистра коэффициента, и предлагаемый вариант на 1,0% меньше базового.
- 6) В наиболее реалистичном сценарии (`gauss_slow`) потребляемая мощность снижена на 4,4% (режим `approx_2`) при $\text{SNR} = 52$ дБ и нулевом систематическом смещении.
- 7) Показано, что эффективность метода монотонно растёт с разрядностью умножителя: накладные расходы вынесенной архитектуры линейны по разрядности, тогда как мощность суммирующего дерева растёт квадратично, поэтому их отношение убывает. Гипотеза подтверждена измерениями на разрядностях 8, 16 и 20 бит.

Перспективы дальнейших исследований:

- Оценка архитектуры на производственных библиотеках стандартных ячеек для получения абсолютных значений мощности, применимых в промышленном контексте.
- Интеграция предложенного умножителя в полную схему N -тапного адаптивного фильтра и оценка системного энергобюджета.
- Исследование масштабирования метода на умножители с большей разрядностью (32, 64 бита) и подтверждение сохранения выявленной монотонной зависимости на этих разрядностях.

Литература

1. Booth A. D. A signed binary multiplication technique // The Quarterly Journal of Mechanics and Applied Mathematics. — 1951. — Vol. 4, № 2.
2. MacSorley O. L. High-speed arithmetic in binary computers // Proceedings of the IRE. — 1961. — Vol. 49, № 1.
3. Tsoumanis K., Axelos N., Moschopoulos N. и др. Pre-encoded multipliers based on non-redundant radix-4 signed-digit encoding // IEEE Transactions on Computers. — 2016. — Vol. 65, № 2.
4. Cui X., Liu W., Chen X., Swartzlander E. A modified partial product generator for redundant binary multipliers // IEEE Transactions on Computers. — 2016. — Vol. 65, № 4.
5. Han J., Orshansky M. Approximate computing: An emerging paradigm for energy-efficient design // Proceedings of the 18th IEEE European Test Symposium (ETS). — 2013.
6. Venkatachalam S., Adams E., Lee H. J., Ko S. B. Design and analysis of area and power efficient approximate booth multipliers // IEEE Transactions on Computers. — 2019. — Vol. 68, № 11.
7. Waris H., Wang C., Liu W. Hybrid low radix encoding-based approximate booth multipliers // IEEE Transactions on Circuits and Systems II: Express Briefs. — 2020. — Vol. 67, № 12.
8. Haider M. H., Zhang H., Ko S. B. Decoder reduction approximation scheme for Booth multipliers // IEEE Transactions on Computers. — 2024. — Vol. 73, № 3.
9. Park G., Kung J., Lee Y. Simplified compressor and encoder designs for low-cost approximate radix-4 booth multiplier // IEEE Transactions on Circuits and Systems II: Express Briefs. — 2023. — Vol. 70, № 3.
10. Zhang T., Jiang H., Mo H. и др. Design of majority logic-based approximate booth multipliers for error-tolerant applications // IEEE Transactions on Nanotechnology. — 2022. — Vol. 21.
11. Zhu X., Li H., Song Y. Exact and approximate Radix-4 recoding multipliers for high-efficiency computation // Microelectronics Journal. — 2025. — Vol. 157. — DOI: 10.1016/j.mejo.2025.106579.

12. Esposito D., Di Meo G., De Caro D. и др. On the use of approximate multipliers in LMS adaptive filters // Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS). — 2018.
13. Mohanty B. K., Patel S. K. Efficient very large-scale integration architecture for variable length block least mean square adaptive filter // IET Signal Processing. — 2015. — Vol. 9, № 8.
14. Haykin S. Adaptive Filter Theory. — 5th ed. — Pearson, 2013. — 907 p.